

Assignment 16.

1. Set up Redux Thunk middleware in a new React app and confirm that your Redux store can now handle async actions by logging a simple async action's output to the console.
2. Create a Redux thunk action called `fetchRestaurants` that fetches a list of restaurants from a public API (such as <https://api.sampleapis.com/restaurants/restaurants>) and stores the data in your Redux store.
3. Build a React component called `RestaurantList` that displays the restaurant names from your Redux store and shows a loading message while the `fetchRestaurants` thunk is in progress.
4. Add error handling to your `fetchRestaurants` thunk so that if the API request fails, an error message is displayed in the `RestaurantList` component.

Hint: Use a separate error state in your Redux slice and display it conditionally in the UI.
5. Refactor your thunk action to fetch only restaurants from a specific city (e.g., 'Ahmedabad') by passing the city as a parameter, and update your UI to allow users to enter a city name and trigger the filtered fetch.

Answer:- 1. Install Dependencies:-

`npm install @reduxjs/toolkit react-redux`

`store.js`

```
import { configureStore } from "@reduxjs/toolkit";
import restaurantReducer from "../restaurantSlice";

export const store = configureStore({
  reducer: {
    restaurants: restaurantReducer,
  },
});
```

`restaurantSlice.js`

```
import {
```

```

    createSlice,
    createAsyncThunk,
  } from "@reduxjs/toolkit";

// Async Thunk
export const fetchRestaurants = createAsyncThunk(
  "restaurants/fetchRestaurants",
  async (city = "", thunkAPI) => {
    try {
      const response = await fetch(
        "https://api.sampleapis.com/restaurants/restaurants"
      );

      if (!response.ok) {
        throw new Error("Failed to fetch restaurants");
      }

      const data = await response.json();

      // Filter by city
      const filteredData = city
        ? data.filter(
            (restaurant) =>
              restaurant.city
                ?.toLowerCase()
                .includes(city.toLowerCase())
          )
        : data;

      return filteredData;
    } catch (error) {
      return thunkAPI.rejectWithValue(
        error.message
      );
    }
  }
);

const restaurantSlice = createSlice({
  name: "restaurants",

  initialState: {
    data: [],
    loading: false,
    error: null,
  },

  reducers: {},

```

```

extraReducers: (builder) => {
  builder

    .addCase(
      fetchRestaurants.pending,
      (state) => {
        state.loading = true;
        state.error = null;
      }
    )

    .addCase(
      fetchRestaurants.fulfilled,
      (state, action) => {
        state.loading = false;
        state.data = action.payload;
      }
    )

    .addCase(
      fetchRestaurants.rejected,
      (state, action) => {
        state.loading = false;
        state.error = action.payload;
      }
    );
  },
});

export default restaurantSlice.reducer;

```

```

export const testThunk = () => {
  return async (dispatch) => {
    setTimeout(() => {
      console.log("Async Action Executed!");
    }, 1000);
  };
};

```

RestaurantList.jsx

```

import { useEffect, useState } from "react";
import {

```

```

    useDispatch,
    useSelector,
  } from "react-redux";

import { fetchRestaurants }
from "../redux/restaurantSlice";

const RestaurantList = () => {
  const dispatch = useDispatch();

  const [city, setCity] = useState("");

  const {
    data,
    loading,
    error,
  } = useSelector(
    (state) => state.restaurants
  );

  useEffect(() => {
    dispatch(fetchRestaurants());
  }, [dispatch]);

  const handleSearch = () => {
    dispatch(fetchRestaurants(city));
  };

  return (
    <div style={{ padding: "20px" }}>
      <h1>Restaurant Finder</h1>

      <input
        type="text"
        placeholder="Enter City"
        value={city}
        onChange={(e) =>
          setCity(e.target.value)
        }
      />

      <button onClick={handleSearch}>
        Search
      </button>

      <hr />

      {loading && (

```

```

        <h3>Loading restaurants...</h3>
      )}

      {error && (
        <h3 style={{ color: "red" }}>
          Error: {error}
        </h3>
      )}

      {!loading &&
        !error &&
        data.map((restaurant) => (
          <div key={restaurant.id}>
            <h4>{restaurant.name}</h4>
          </div>
        ))}
    </div>
  );
};

export default RestaurantList;

```

App.jsx

```

import RestaurantList from "../components/RestaurantList";

function App() {
  return (
    <div>
      <RestaurantList />
    </div>
  );
}

export default App;

```

main.jsx

```

import React from "react";
import ReactDOM from "react-dom/client";

import App from "../App";

import { Provider } from "react-redux";
import { store } from "../redux/store";

```

```
ReactDOM.createRoot(  
  document.getElementById("root")  
)  
.render(  
  <Provider store={store}>  
    <App />  
  </Provider>  
)  
;
```